



UNIVERSIDAD AUTÓNOMA DE NUEVO LEÓN
Facultad de Ciencias Físico Matemáticas



Base de Datos

Grupo 033

Emilio Martinez Verduzco
2086026

Stored Procedure y Trigger en MySQL

24/04/2026

Caso 1 - Registro de venta con Stored Procedure

Resumen

Una tienda de artículos de oficina presenta problemas al registrar ventas porque el proceso se realiza mediante varias instrucciones separadas. Esto puede provocar errores como ventas incompletas, inventarios incorrectos o totales mal calculados. Para solucionar esta situación se propone utilizar un Stored Procedure, que permita validar la información y ejecutar todo el proceso de venta dentro de una sola operación controlada, garantizando la consistencia de los datos.

Parámetros de entrada

- ID del cliente
- ID del producto
- Cantidad
- Usuario que registra la venta

Son parámetros porque cambian en cada venta y deben ser proporcionados por el usuario o la aplicación.

Nombre del procedimiento

sp_registrar_venta_simple

Variables internas

- v_stock → almacena el inventario disponible.
- v_precio → almacena el precio del producto.
- v_subtotal → almacena el total calculado.
- v_venta_id → almacena el ID generado para la venta.

Validaciones lógicas

1. Verificar que el cliente exista.
2. Verificar que el producto exista y esté activo.
3. Verificar que la cantidad sea mayor a cero.
4. Verificar que exista inventario suficiente.

Uso de START TRANSACTION, COMMIT y ROLLBACK

Permiten que todo el proceso se ejecute como una sola unidad. Si ocurre algún error se utiliza ROLLBACK para cancelar los cambios; si todo sale bien se utiliza COMMIT para guardar la información.

Secuencia del proceso

1. Consultar stock y precio.
2. Validar información.
3. Calcular subtotal.
4. Insertar encabezado de venta.
5. Obtener ID de la venta.
6. Insertar detalle de venta.
7. Actualizar inventario.
8. Confirmar transacción.

¿Qué pasa si se actualiza el inventario primero?

Si ocurre un error después de descontar el inventario, el stock quedaría reducido aunque la venta no exista, generando inconsistencias en la base de datos.

Procedimiento en MySQL

```
1  DELIMITER $$
2
3  CREATE PROCEDURE sp_registrar_venta_simple(
4      IN p_cliente_id INT,
5      IN p_producto_id INT,
6      IN p_cantidad INT,
7      IN p_usuario VARCHAR(100)
8  )
9  BEGIN
10
11      DECLARE v_stock INT;
12      DECLARE v_precio DECIMAL(10,2);
13      DECLARE v_total DECIMAL(10,2);
14      DECLARE v_venta_id INT;
15
16      -- Verificar cliente activo
17      IF NOT EXISTS (
18          SELECT 1
19          FROM clientes
20          WHERE cliente_id = p_cliente_id
21              AND activo = 1
22      ) THEN
23          SIGNAL SQLSTATE '45000'
24          SET MESSAGE_TEXT = 'Cliente no existe o esta inactivo';
25      END IF;
```

```

27  -- Obtener stock y precio del producto
28  SELECT stock_actual, precio_venta
29  INTO v_stock, v_precio
30  FROM productos
31  WHERE producto_id = p_producto_id
32  AND activo = 1;
33
34  -- Validar cantidad
35  IF p_cantidad <= 0 THEN
36      SIGNAL SQLSTATE '45000'
37      SET MESSAGE_TEXT = 'Cantidad invalida';
38  END IF;
39
40  -- Validar inventario
41  IF v_stock < p_cantidad THEN
42      SIGNAL SQLSTATE '45000'
43      SET MESSAGE_TEXT = 'Inventario insuficiente';
44  END IF;
45
46  START TRANSACTION;
47
48  SET v_total = v_precio * p_cantidad;
49
50  -- Insertar venta
51  INSERT INTO ventas(
52      fecha_venta,
53      cliente_id,
54      total,
55      usuario_registro
56  )
57  VALUES(
58      NOW(),
59      p_cliente_id,
60      v_total,
61      p_usuario
62  );
63
64  SET v_venta_id = LAST_INSERT_ID();
65
66  -- Insertar detalle
67  INSERT INTO detalle_venta(
68      venta_id,
69      producto_id,
70      cantidad,
71      precio_unitario,
72      subtotal
73  )
74  VALUES(
75      v_venta_id,
76      p_producto_id,
77      p_cantidad,
78      v_precio,
79      v_total
80  );
81
82  -- Actualizar inventario
83  UPDATE productos
84  SET stock_actual = stock_actual - p_cantidad
85  WHERE producto_id = p_producto_id;
86
87  COMMIT;
88
89  END$$
90
91  DELIMITER ;

```

Pruebas realizadas

SQLQuery3.s...emimt (82))* Options

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17

-- PRUEBA CORRECTA

EXEC sp_registrar_venta_simple
@cliente_id = 1,
@producto_id = 1,
@cantidad = 2,
@usuario = 'equipo_1';

-- PRUEBA CON ERROR

EXEC sp_registrar_venta_simple
@cliente_id = 1,
@producto_id = 1,
@cantidad = 100,
@usuario = 'equipo_1';

82 % 2 0

Messages

Msg 50000, Level 16, State 1, Procedure sp_registrar_venta_simple, Line 49 [Batch Start Line 0]
Inventario insuficiente.

Completion time: 2026-06-08T12:49:26.1412182-06:00

Consultas de verificación

SQLQuery3.s...emimt (82))* Options

1
2
3
4
5

SELECT * FROM ventas;

SELECT * FROM detalle_venta;

SELECT * FROM productos;

82 % 3 0

Results Messages

	venta_id	fecha_venta	cliente_id	total	usuario_registro
1	1	2026-06-08 12:49:26.130	1	2500.00	equipo_1

	detalle_id	venta_id	producto_id	cantidad	precio_unitario	subtotal
1	1	1	1	2	1250.00	2500.00

	producto_id	nombre	categoria	precio_venta	stock_actual	stock_minimo	activo
1	1	Tóner HP 17A	Consumibles	1250.00	10	6	1
2	2	Paquete de hojas carta	Papelería	95.00	40	10	1
3	3	Mouse inalámbrico	Accesorios	280.00	18	5	1
4	4	Teclado USB	Accesorios	350.00	14	4	1
5	5	Memoria USB 64GB	Almacenamiento	190.00	25	8	1
6	6	Carpeta tamaño carta	Papelería	25.00	60	15	1
7	7	Monitor 24 pulgadas	Equipo	2899.00	7	3	1
8	8	Producto inactivo demo	Demo	100.00	10	2	0

Preguntas de discusión

¿Por qué no usar un Trigger como solución principal?

Porque el registro de ventas es un proceso que debe ejecutarse cuando el usuario lo solicite y requiere parámetros de entrada, validaciones y transacciones.

¿Qué ventaja tiene centralizar la lógica en un procedimiento?

Facilita el mantenimiento, evita duplicar código y garantiza que todas las ventas sigan las mismas reglas.

¿Qué parámetros usarían y por qué?

ID del cliente, ID del producto, cantidad y usuario, porque son datos que cambian en cada operación.

¿Qué validación debe ejecutarse antes de descontar inventario?

Verificar que exista suficiente stock disponible.

¿Qué papel cumple LAST_INSERT_ID()?

Permite obtener el ID de la venta recién creada para relacionarla con el detalle de venta.

¿Qué evidencia demostraría que funcionó correctamente?

La creación de registros en las tablas ventas y detalle_venta, junto con la actualización correcta del inventario.

¿Cómo comprobarían que la transacción protege la integridad de los datos?

Provocando un error durante el proceso y verificando que ningún cambio se haya guardado en la base de datos gracias al ROLLBACK.

Conclusión:

Un Stored Procedure es útil cuando se necesita ejecutar un proceso de negocio con varias validaciones y pasos relacionados, como registrar ventas, actualizar inventario o generar facturas, asegurando que toda la información se procese de forma correcta y consistente.

Caso 2 - Alerta automática de inventario mínimo con Trigger

Resumen

La tienda presenta problemas para identificar a tiempo los productos que están por debajo del inventario mínimo, ya que la revisión se realiza manualmente. Esto puede provocar faltantes de mercancía y retrasos en la reposición de productos. Para solucionar esta situación se propone utilizar un Trigger, que permita generar alertas automáticamente cuando el stock de un producto quede por debajo del nivel mínimo establecido.

¿Por qué no conviene usar un Stored Procedure?

Porque la alerta debe generarse automáticamente cuando cambie el inventario. Si dependiera de un Stored Procedure, el usuario tendría que ejecutarlo manualmente cada vez, aumentando el riesgo de olvidos y errores.

Tabla y evento del Trigger

- Tabla: productos
- Evento: UPDATE

¿BEFORE UPDATE o AFTER UPDATE?

AFTER UPDATE, porque permite trabajar con los datos ya actualizados y verificar si el inventario quedó por debajo del mínimo después del cambio.

Uso de OLD y NEW

- OLD.stock_actual: valor del stock antes de la actualización.
- NEW.stock_actual: valor del stock después de la actualización.
- NEW.stock_minimo: valor mínimo permitido para el producto.

Condición lógica

Generar la alerta cuando el stock pase de estar en un valor igual o superior al mínimo a un valor inferior al mínimo.

INSERT de la alerta

Insertar en la tabla alertas_inventario:

- producto_id
- stock_actual
- stock_minimo
- fecha_alerta
- mensaje
- atendida

Codigo del Trigger

```
1  CREATE TRIGGER trg_alerta_inventario
2  ON productos
3  AFTER UPDATE
4  AS
5  BEGIN
6
7      INSERT INTO alertas_inventario
8      (
9          producto_id,
10         stock_actual,
11         stock_minimo,
12         fecha_alerta,
13         mensaje,
14         atendida
15     )
16     SELECT
17         i.producto_id,
18         i.stock_actual,
19         i.stock_minimo,
20         GETDATE(),
21         'Producto por debajo del stock minimo',
22         0
23     FROM inserted i
24     INNER JOIN deleted d
25         ON i.producto_id = d.producto_id
26     WHERE
27         d.stock_actual >= d.stock_minimo
28         AND i.stock_actual < i.stock_minimo;
29
30 END;
31 GO
```


Prueba Update sin Alerta

1	
2	UPDATE productos
3	SET stock_actual = 11
4	WHERE producto_id = 1;
5	
6	SELECT * FROM alertas_inventario;

12 % ✓ No issues found

Results Messages

alerta_id	producto_id	stock_actual	stock_minimo	fecha_alerta	mensaje	atendida
-----------	-------------	--------------	--------------	--------------	---------	----------

Prueba Update con Alerta

1	
2	UPDATE productos
3	SET stock_actual = 5
4	WHERE producto_id = 1;
5	
6	SELECT * FROM alertas_inventario;

82 % ✓ No issues found

Results Messages

	alerta_id	producto_id	stock_actual	stock_minimo	fecha_alerta	mensaje	atendida
1	1	1	5	6	2026-06-08 13:17:12.690	Producto por debajo del stock minimo	0

¿Cuál es el evento exacto que activa el Trigger?

El evento UPDATE sobre la tabla productos.

¿Qué diferencia existe entre OLD.stock_actual y NEW.stock_actual?

OLD contiene el valor anterior del stock y NEW contiene el valor actualizado.

¿Por qué AFTER UPDATE es más adecuado?

Porque permite trabajar con los valores ya actualizados y verificar el estado final del inventario.

¿Qué pasaría si no se controla la generación repetida de alertas?

Se generarían múltiples registros innecesarios para el mismo producto.

¿Qué ventajas ofrece un Trigger frente a la revisión manual?

Detecta problemas inmediatamente, reduce errores humanos y automatiza el monitoreo.

Dos casos reales donde usarías un Trigger

- Registro automático de auditorías.
- Historial de movimientos bancarios o financieros.

Conclusión:

Un Trigger es útil cuando se necesita ejecutar acciones automáticas ante cambios en la base de datos, como alertas o auditorías. Sin embargo, si se abusa de ellos pueden volver más complejo el sistema, dificultar el mantenimiento y afectar el rendimiento de la base de datos